Instructor Notes

Preliminary steps for lecturer

Basic content

- Open repoositry (add link)
- Start GitHub Codespace
- Open actionlint playground
- Go through the GitHub Action settings for forks

Setting up a First GitHub Action

- Create a new folder `.github/workflows/`

```
$ mkdir -p .github/workflows
$ tree
```

- Create new workflow file

```
$ cd .github/workflows/
$ touch first_workflow.yml
```

  and open in editor.

- Add content to file

```yaml
name: My first GitHub workflow
run-name: First workflow # Optional

# Events triggering the workflow
on: [push]

# Group of jobs to run as poart of the workflow
jobs:
  greet:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Hello world"
```

- Add file to repository, commit, and push

```
$ git add first_workflow.yml
$ git commit -m "My first workflow"
$ git push
```

- Go to `Actions` tab of the repository.

  The workflow should run on GitHub (maybe is already finished).

- Inspect the output. There should be three steps

  1. `Set up job`
  2. `Run echo "Hello world"`
  3. `Complete job`

Steps 1 and 2 are implicitly added by GitHub.

Adding a timeout

- Update the job section to have a timeout

```
  jobs:
    greet:
      runs-on: ubuntu-latest
      timeout-minutes: 5
      steps:
        - run: echo "Hello world"
```

Running own code

- Add another step to run our own code in the `jobs` section

```
  run-code:
    runs-on: ubuntu-latest
    timeout-minutes: 5
    steps:
      - uses: actions/checkout@v5

      - uses: actions/setup-python@v6
        with:
          python-version: '3.14'

      - run: python mycode.py
```

- Inspect the workflow run in the “Actions” tab
  - There are two jobs run.
  - The jobs run in parallel
- Full workflow configuration after this section

```
name: My first GitHub workflow
run-name: First workflow # Optional

# Events triggering the workflow
on: [push]

# Group of jobs to run as part of the workflow
jobs:
  greet:
    runs-on: ubuntu-latest
    timeout-minutes: 5
    steps:
      - run: echo "Hello world"

  run-code:
    runs-on: ubuntu-latest
    timeout-minutes: 5
    steps:
      - uses: actions/checkout@v5
```

```

      - uses: actions/setup-python@v6
        with:
          python-version: '3.14'

      - run: python code.py
```

Testing own code

- Add two new steps to install the dependencies and run the test

```
      - name: Install test dependencies
        run: |
          python -m pip install --upgrade pip
          python -m pip install -r requirements-dev.txt

      - name: "Run tests"
        run: pytest
```

  We can have several commands within a `run` statement!

- Step through the Action output and check what was printed

Optional: Adding a Failing step

- Change a test such that it fails and commit
- Browse through the repository to see how a failing test looks like

Refining the workflow

Exploring settings related to GitHub Actions

- Enable CodeQL
  - Repository must be **`public`** or part of an organisation that has GitHub Advanced Security (GHAS) features.
- Go through settings for GitHub Actions

Reducing permissions

- Check out the security tab of the repository. There should be one warning about having too broad permissions.
- GitHub generates a `GITUB_TOKEN` witin the job that can be used to carry out tasks on behalf of the user triggering the workflow. You want to restrict this such that workflows cannot do anyting bad.
- We can restrict permissions within the workflow and grant it to jobs as they need them.
- Add

```
run-name: First workflow # Optional
permissions: {}
```

and

```yaml
run-code:
  runs-on: ubuntu-latest
  timeout-minutes: 5
  permissions:
    contents: read
```

This jobs does not need any additional permissions if the repository is **`public`**, but it is needed if the repository is `internal` or **`private`**.

Matrix Builds

- Let's run and test our code on several Python versions
- Add a `strategy` section to the `run-code` job

```yaml
permissions:
  contents: read

strategy:
  matrix:
    python-version: [ "3.12", "3.13", "3.14" ]
```

and use the python version when setting up Python

```yaml
  - uses: actions/setup-python@v6
    with:
      python-version: ${{ matrix.python-version }}
```

- Go to the `Actions` tab on the repository and explore the matrix build. It does look slightly diffrent than before.

Additional operating systems GitHub Action provides runners for different operatin systems. We can also use this within the matrix strategy. Useful for multi-platform software and software packaging.

- Update the configuration to read

```yaml
run-code:
  needs: checks
  runs-on: ${{ matrix.os }}
  timeout-minutes: 5
  permissions:
    contents: read

  strategy:
    fail-fast: false
    matrix:
      python-version: [ "3.12", "3.13", "3.14" ]
      os: [ "macos-latest", "windows-latest", "ubuntu-latest" ]
```

Note: This increases the number of jobs to 9. Be cautious when adding more parameters inthe matrix to avoid unnecessarily long workflow runs.

Actionlint

- Open actionlint playground
- Copy and paste our action and check for suggested improvements

Conditionals

- We can make jobs and steps conditional.
- Add an additional step to the `run-code` job

```
  - name: "Extra step for Python 3.14"
    if: ${{ matrix.python-version == '3.14' }}
    run: echo "Extra step for Python 3.14"
```

and commit to the repository. This step will only be executed if Python 3.14 is used.

Note: It is important to use single quotes `'` in the conditional to ensure the correct evaluation.

Job dependencies

- We can make steps depend on each other, e.g., to not waste resources (compute minutes) of jobs.
- We can add an additional job for (theoretical) checks, e.g. style checks. Add the followuing to the action

```
checks:
  runs-on: ubuntu-latest
  timeout-minutes: 5
  steps:
    - name: Run checks
      run: |
        echo "Run checks"
        sleep 20

run-code:
  needs: style
  runs-on: ubuntu-latest
```

to have the `run-code` job execute after the `checks` job. The `checks` job must succeed for the `run-code` job to start.

- Check the `Actions` tab in the repository and see how the job dependencies are indicated.

Workflow triggers

- Only run on push to `main` and pull requests targetting `main`

  Replace

```
on: [push]
```

by

```
on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]
```

- Create a new branch and add a new file

```
git checkout -b newbranch
touch somefile.txt
git add somefile.txt
git commit -m "New file"
git push
```

- Check repository website and see that workflow is not running

Workflows in pull request

- Creat a pull request from the pushed branch
- Verify that the jobs start running in the PR

Pre-made GitHub Actions / The GitHub Action Marketplace

- Navigate to https://github.com/marketplace and choose `Actions` -> `All Actions` on the left.
- Pin GitHub Action to a certain commit hash

Further topics (Optional)

The topics below are optional and may be covered depending on questions and time. There is no checkpoint available for this.

Billing

- Billing and usage
  - Check the "Billing" section in the user account.
- Security hardening of Actions

Slack integration

One can send messages from GitHub to Slack. This can be helpful for monitoring of failing jobs.

Creating Downloadable Artifacts

- Workflow artifacts

```
  - name: "Create new file"
    run: echo "Hello world!" >> new_file.txt

  - name: "Upload Artifact"
    uses: actions/upload-artifact@v4
    with:
      name: my-artifact
      path: new_file.txt
      retention-days: 5
```

- A similar action is available for downloading artifacts in a job. This can be used to share data between jobs.

Running Action within a Docker container

- Using containerized services

```yaml
jobs:
  # Label of the container job
  container-job:
    # Containers must run in Linux based operating systems
    runs-on: ubuntu-latest
    # Docker Hub image that `container-job` executes in
    container: python:3.14-slim
```

- This will not contain the the same preinstalled software that you get on a GitHub runner!